



Anomaly Detection in Time Series – Regression-Based Methods

Regression-Based Anomaly Detection – `hanr_ml()`

The Regression Approach

``hanr_ml()`` detects anomalies by training a regression model to forecast the next value of a time series from a sliding window of past observations. At inference time, the model predicts each step and computes the residual (prediction error). Observations whose residuals exceed a learned threshold are flagged as anomalies.

The `hanr_ml()` wrapper

- Accepts any DALToolbox regression model (`ts_elm`, `ts_mlp`, `ts_rf`, `ts_svm`, `ts_lstm`, `ts_conv1d`)
- No labeled anomalies required — trained on normal behavior only
- Anomaly score = absolute prediction residual at each time step
- Residuals above a threshold (estimated from training) → flagged as anomalies
- ``sw_size``: sliding window size (default 15); controls how much history is used

Step-by-step workflow

- 1. Normalize the series: ``ts_norm_gminmax()`` (global min-max)
- 2. Wrap a regressor: ``hanr_ml(ts_elm(...))``, ``hanr_ml(ts_mlp(...))``, etc.
- 3. Fit on the full series: ``model <- fit(model, dataset$serie)``
- 4. Detect: ``detection <- detect(model, dataset$serie)``
- 5. Evaluate: ``evaluate(model, detection$event, dataset$event)``
- 6. Visualize: ``har_plot()`` for series + residuals + threshold

Challenges

- Model selection: different regressors yield very different results
- Threshold sensitivity: too low → false positives; too high → false negatives
- Non-stationarity: model trained on one regime may fail on another
- Deep models (LSTM, Conv1D) require more epochs and ``daltoolboxdp``

Supported regressors (`tspredict` / `daltoolboxdp`)

- Classical: ``ts_elm()``, ``ts_mlp()``, ``ts_rf()``, ``ts_svm()``
- Deep learning: ``ts_lstm()``, ``ts_conv1d()`` (requires ``daltoolboxdp``)
- All share the same ``fit()`` / ``detect()`` interface via ``hanr_ml()``

📄 ``hanr_ml()`` is a unified wrapper: swap the inner regressor to compare ELM, MLP, RF, SVM, LSTM, and Conv1D without changing any other part of the pipeline. This makes systematic comparison straightforward.

📖 Hyndman, R. J., & Athanasopoulos, G. (2021). *Forecasting: Principles and Practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>

📖 Ogasawara, E., Salles, R., Porto, F., & Pacitti, E. (2025). *Event Detection in Time Series* (1st ed.). Springer Nature Switzerland. <https://doi.org/10.1007/978-3-031-75941-3>

Extreme Learning Machine (ELM)

What is ELM?

ELM is a single-hidden-layer feedforward neural network where the input weights and biases are randomly assigned and never updated. Only the output weights are learned — via a simple least-squares solution. This makes training extremely fast compared to backpropagation-based networks.

Architecture

- Input layer → randomly weighted hidden layer → output layer
- Hidden weights: fixed random values (not trained)
- Output weights: computed analytically (Moore-Penrose pseudoinverse)
- Activation function: configurable (e.g., purelin, sigmoid, ReLU)

Regression-based detection

- ELM is trained on normal time series behavior
- At inference, it predicts the next value from a sliding window
- Prediction error (residual) is computed at each step
- Residuals above a learned threshold are flagged as anomalies

Key parameters (in `tspredict`)

- `input_size`: length of the sliding window
- `nhid`: number of hidden neurons
- `actfun`: activation function ("purelin" = linear)
- Normalization: `ts_norm_gminmax()` recommended

Strengths

- Extremely fast training (no iterative optimization)
- Simple to implement and tune
- Good baseline for regression-based anomaly detection

Limitations

- Random weights may lead to suboptimal representations
- Less accurate than MLP or SVM on complex patterns
- Sensitive to the number of hidden neurons

📖 ELM trades accuracy for speed. The fixed random hidden layer avoids the vanishing gradient problem and makes training $O(n)$ — ideal for quick experiments before committing to heavier models.

📖 Huang, G.-B., Zhu, Q.-Y., & Siew, C.-K. (2006). Extreme learning machine: Theory and applications. *Neurocomputing*, 70(1–3), 489–501. <https://doi.org/10.1016/j.neucom.2005.12.126>

15 · hanr_ml + ELM – Extreme Learning Machine

```
library(daltoolbox)
library(harbinger)
library(tspredit)

data(examples_anomalies)
dataset <- examples_anomalies$simple

har_plot(harbinger(), dataset$serie)

# ELM como regressor
model <- hanr_ml(
  ts_elm(ts_norm_gminmax(),
    input_size = 4,
    nhid = 3,
    actfun = "purelin"))
model <- fit(model, dataset$serie)

detection <- detect(model,
  dataset$serie)
# Resultado: idx=52 (1 FN, 1 FP)

print(detection |>
  dplyr::filter(event == TRUE))
## idx event type
## 1 52 TRUE anomaly
```

```
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  0   1
##  FALSE  1  99

har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step-by-step – ELM

- Requires the `tspredit` package
- `ts_norm_gminmax()`: global min-max normalization
- `input_size`: sliding window length (4)
- `nhid`: hidden neurons with fixed random weights
- ELM predicts the next value; large errors flag anomalies
- Detected idx=52 instead of 50: 1 FN, 1 FP

📌 **Key concept:** `hanr_ml()` is a wrapper that accepts any DALToolbox regressor. ELM uses fixed random weights in the hidden layer — training is extremely fast, but it may be less accurate than MLP.

Multilayer Perceptron (MLP)

What is MLP?

MLP is a fully connected feedforward neural network with one or more hidden layers. Unlike ELM, all weights are learned through backpropagation and gradient descent. This allows MLP to learn complex non-linear mappings between inputs and outputs.

Architecture

- Input layer → one or more hidden layers → output layer
- All weights updated iteratively via backpropagation
- Activation functions: sigmoid, tanh, ReLU (hidden); linear (output for regression)
- L2 regularization (weight decay) prevents overfitting

Regression-based detection

- MLP is trained to predict the next value from a sliding window of past values
- Prediction residuals are computed at each time step
- A threshold (learned from training residuals) separates normal from anomalous
- Large residuals → anomaly flagged

Key parameters (in `tspredict`)

- `input_size`: sliding window length (number of past steps)
- `size`: number of hidden neurons
- `decay`: L2 regularization coefficient (0 = no regularization)
- Normalization: `ts_norm_gminmax()` recommended

Strengths

- Learns non-linear patterns through backpropagation
- More accurate than ELM on structured time series
- Flexible architecture (depth, width, activation)

Limitations

- Slower training than ELM (iterative optimization)
- Sensitive to hyperparameters (size, decay, learning rate)
- May overfit on very small datasets

❏ MLP with backpropagation achieved a perfect result (1 TP, 0 FP) on the simple dataset — outperforming ELM. The key difference: learned weights adapt to the data distribution, not random projections.

📖 Rumelhart, D. E., Hinton, G. E., & Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323, 533–536. <https://doi.org/10.1038/323533a0>

16 · hanr_ml + MLP – Multilayer Perceptron

```
library(daltoolbox)
library(harbinger)
library(tspredit)

data(examples_anomalies)
dataset <- examples_anomalies$simple

# MLP como regressor
model <- hanr_ml(
  ts_mlp(ts_norm_gminmax(),
    input_size = 3,
    size = 3,
    decay = 0))
model <- fit(model, dataset$serie)

detection <- detect(model,
  dataset$serie)

print(detection |>
  dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly
```

```
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##  FALSE  0 100

har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step-by-step – MLP

- size: number of hidden units (3)
- decay: L2 regularization strength (0 = no regularization)
- input_size: sliding window of 3 previous steps
- MLP predicts the next value in the window; large residuals flag anomalies
- Perfect result: 1 TP, 0 FP

📌 **Key concept:** MLP with backpropagation learns non-linear mappings by updating weights across all layers. Compared to ELM, training is slower — but it is generally more accurate for this type of task.

Random Forest (Regression)

What is Random Forest?

Random Forest is an ensemble method that builds multiple decision trees on random subsets of the training data and features, then aggregates their predictions (average for regression). The randomness reduces variance and improves generalization compared to a single decision tree.

Architecture

- Ensemble of `ntree` independent decision trees
- Each tree trained on a bootstrap sample of the data
- At each split, only a random subset of features is considered
- Final prediction: average of all tree outputs (regression)

Regression-based detection

- RF is trained to predict the next value from a sliding window
- Prediction residuals are computed at each time step
- Residuals above a threshold are flagged as anomalies
- Sensitive to local variations — may produce false positives

Key parameters (in `tspredict`)

- `input_size`: sliding window length
- `ntree`: number of trees in the forest
- `nodesize`: minimum terminal node size (controls tree depth)
- Normalization: `ts_norm_gminmax()` recommended

Strengths

- Robust to outliers and noise in training data
- No need for feature scaling (but normalization still recommended)
- Naturally handles non-linear relationships

Limitations

- Can be sensitive to local variations → false positives
- Slower inference than ELM or SVM
- Less interpretable than a single decision tree

❏ On the simple dataset, RF produced 2 false positives (`idx=75, 84`) and missed the true anomaly. This illustrates that ensemble methods are not always superior — tuning `ntree` and `nodesize` is critical.

📖 Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32.

<https://doi.org/10.1023/A:1010933404324>

17 · hanr_ml + Random Forest – Forest Regression

```
library(daltoolbox)
library(harbinger)
library(tspredit)

data(examples_anomalies)
dataset <- examples_anomalies$simple

har_plot(harbinger(), dataset$serie)

# Random Forest como regressor
model <- hanr_ml(
  ts_rf(ts_norm_gminmax(),
    input_size = 4,
    nodesize = 1,
    ntree = 20))
model <- fit(model, dataset$serie)

detection <- detect(model,
  dataset$serie)
# Resultado: idx=75,84 (0 TP, 2 FP)

print(detection |>
  dplyr::filter(event == TRUE))
## idx event type
## 1 75 TRUE anomaly
## 2 84 TRUE anomaly
```

```
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  0   2
##  FALSE  1  98

har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step-by-Step – Random Forest

- `nodesize`: minimum terminal node size (1)
- `ntree`: number of trees in the forest (20)
- RF detected idx=75 and 84 (false positives on this dataset)
- Demonstrates that RF can be sensitive to local variations
- Comparing methods is essential before making a final choice

 **Key concept:** Random Forest used as a forecasting regressor. On this dataset, it produced 2 FPs — sensitive to local variations. Tuning `ntree` and `nodesize` may improve precision.

Support Vector Machine (SVR)

What is SVR?

Support Vector Regression (SVR) extends the SVM framework to regression tasks. It finds a function that deviates from the actual target values by at most ϵ (epsilon), while being as flat as possible. The kernel trick allows SVR to model non-linear relationships by mapping inputs into a high-dimensional feature space.

Architecture

- Maps input features into a high-dimensional space via a kernel function
- Finds the optimal hyperplane (regression function) within an ϵ -tube
- Support vectors: training points that lie on or outside the ϵ -tube
- RBF kernel: $K(x, x') = \exp(-\gamma \|x - x'\|^2)$ — captures non-linear patterns

Regression-based detection

- SVR is trained to predict the next value from a sliding window
- Prediction residuals are computed at each time step
- Residuals above a threshold are flagged as anomalies
- RBF kernel captures non-linear temporal patterns effectively

Key parameters (in `tspredict`)

- `input_size`: sliding window length
- `kernel`: kernel function ("radial" = RBF recommended)
- `cost (C)`: penalty for residuals outside the ϵ -tube
- Normalization: `ts_norm_gminmax()` required for SVM

Strengths

- Effective in high-dimensional spaces
- Robust to overfitting (controlled by C and ϵ)
- RBF kernel handles non-linear temporal patterns
- Perfect result on the simple dataset (1 TP, 0 FP)

Limitations

- Requires feature normalization
- Kernel and hyperparameter selection can be non-trivial
- Slower than ELM; not ideal for very large datasets

 **Key concept:** SVR with RBF kernel achieved a perfect score on the simple dataset — matching MLP and Conv1D. Its ability to map inputs into a high-dimensional space makes it particularly effective for non-linear time series patterns.

 Smola, A. J., & Schölkopf, B. (2004). A tutorial on support vector regression. *Statistics and Computing*, 14(3), 199–222.
<https://doi.org/10.1023/B:STCO.0000035301.49549.88>

18 · hanr_ml + SVM – Support Vector Machine

```
library(daltoolbox)
library(harbinger)
library(tspredit)

data(examples_anomalies)
dataset <- examples_anomalies$simple

# SVM como regressor
model <- hanr_ml(
  ts_svm(ts_norm_gminmax(),
    input_size = 4,
    kernel = "radial"))
model <- fit(model, dataset$serie)

detection <- detect(model,
  dataset$serie)
```

```
print(detection |>
  dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly

evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   0
##  FALSE  0 100

har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step-by-step – SVM

- `kernel = "radial"`: RBF kernel to capture non-linear relationships
- `input_size = 4`: sliding window of 4 previous time steps
- SVM predicts the next value; large residuals are flagged as anomalies
- Perfect result: 1 TP, 0 FP on this dataset
- Contrast with RF: SVM was more precise on this dataset

📌 **Key concept:** SVM with an RBF kernel maps inputs into a high-dimensional space, capturing non-linear patterns. It achieved a perfect score on this dataset — a notable contrast to Random Forest, which produced false positives.

Long Short-Term Memory (LSTM)

What is LSTM?

LSTM is a recurrent neural network (RNN) architecture designed to capture long-range temporal dependencies. It introduces a memory cell and three gating mechanisms (input, forget, output) that control the flow of information across time steps, solving the vanishing gradient problem of standard RNNs.

Architecture

- Memory cell: stores information across many time steps
- Forget gate: decides what to discard from the cell state
- Input gate: decides what new information to store
- Output gate: decides what to output from the cell state
- Trained end-to-end via backpropagation through time (BPTT)

Regression-based detection

- LSTM is trained to predict the next value in the series
- Captures long-range dependencies that simpler models miss
- Prediction residuals are computed at each time step
- Residuals above a threshold are flagged as anomalies
- Requires `daltoolboxdp` package in addition to `tspredict`

Key parameters (in `tspredict/daltoolboxdp`)

- `input_size`: length of the input sequence window
- `epochs`: number of training iterations (10,000 recommended)
- Normalization: `ts_norm_gminmax()` required
- Training is significantly slower than classical models

Strengths

- Captures long-range temporal dependencies
- Well-suited for complex, non-stationary time series
- State-of-the-art for many sequence modeling tasks

Limitations

- Very slow training (requires many epochs)
- Needs more data to generalize well
- May produce false positives on simple datasets (1 TP, 1 FP observed)
- Overkill for locally-structured anomalies (Conv1D may be better)

❏ LSTM is powerful but not always the best choice. On the simple dataset it produced 1 FP — Conv1D achieved a perfect result with similar training cost. Reserve LSTM for series with genuine long-range dependencies.

📖 Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780.

<https://doi.org/10.1162/neco.1997.9.8.1735>

19 · hanr_ml + LSTM – Long Short-Term Memory

```
library(daltoolbox)
library(harbinger)
library(tspredit)
library(daltoolboxdp)

data(examples_anomalies)
dataset <- examples_anomalies$simple

har_plot(harbinger(), dataset$serie)

# LSTM como regressor
model <- hanr_ml(
  ts_lstm(ts_norm_gminmax(),
    input_size = 4,
    epochs = 10000))
model <- fit(model, dataset$serie)

detection <- detect(model,
  dataset$serie)
# Resultado: idx=50,54 (1 TP, 1 FP)

print(detection |>
  dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly
## 2 54 TRUE anomaly
```

```
evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
##   TRUE  1   1
##  FALSE  0  99

har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step-by-step – LSTM

- Requires `daltoolboxdp` in addition to `tspredit`
- LSTM captures long-range temporal dependencies in the series
- `epochs = 10000`: intensive training is necessary for convergence
- Result: 1 TP, 1 FP (idx=50 and 54 detected)
- Slower than classical models, but capable of capturing complex temporal patterns

 **Key concept:** LSTM maintains an internal state across the sequence, enabling it to capture long-range dependencies. This makes it well-suited for series with complex temporal patterns, though it requires more data and significantly longer training time.

1D Convolutional Network (Conv1D)

What is Conv1D?

A 1D Convolutional Neural Network applies learned filters (kernels) along the time axis of a sequence. Each filter detects a specific local pattern regardless of its position in the series (translation invariance). Stacking convolutional layers allows the network to learn hierarchical temporal features.

Architecture

- Convolutional layers: apply sliding filters over the time window
- Pooling layers (optional): reduce dimensionality, increase receptive field
- Fully connected layers: map extracted features to the output
- Translation invariance: same pattern detected anywhere in the series
- Trained via backpropagation (same as MLP)

Regression-based detection

- Conv1D is trained to predict the next value from a sliding window
- Convolutional filters capture local temporal patterns (shape, slope, spike)
- Prediction residuals are computed at each time step
- Residuals above a threshold are flagged as anomalies
- Requires `daltoolboxdp` package in addition to `tspredict`

Key parameters (in `tspredict/daltoolboxdp`)

- `input_size`: length of the input window (= filter receptive field)
- `epochs`: number of training iterations (10,000 recommended)
- Normalization: `ts_norm_gminmax()` required
- Filter size and number of filters defined internally

Strengths

- Translation invariance: detects patterns regardless of position
- More efficient than LSTM for locally-structured anomalies
- Perfect result on the simple dataset (1 TP, 0 FP)
- Parallelizable training (unlike RNNs)

Limitations

- Limited receptive field — may miss very long-range dependencies
- Requires `daltoolboxdp` (deep learning extension)
- Slower training than classical models (ELM, SVM)

📄 Conv1D outperformed LSTM on the simple dataset with the same training budget. When anomalies are defined by local shape (spike, step change), Conv1D is the preferred deep learning choice over LSTM.

📖 Kiranyaz, S., et al. (2021). 1D convolutional neural networks and applications: A survey. *Mechanical Systems and Signal Processing*, 151, 107398. <https://doi.org/10.1016/j.ymssp.2020.107398>

20 · hanr_ml + Conv1D – 1D Convolutional Network

```
library(daltoolbox)
library(harbinger)
library(tspredit)
library(daltoolboxdp)

data(examples_anomalies)
dataset <- examples_anomalies$simple

# Conv1D como regressor
model <- hanr_ml(
  ts_conv1d(ts_norm_gminmax(),
    input_size = 4,
    epochs = 10000))
model <- fit(model, dataset$serie)
```

```
detection <- detect(model,
  dataset$serie)

print(detection |>
  dplyr::filter(event == TRUE))
## idx event type
## 1 50 TRUE anomaly

evaluation <- evaluate(model,
  detection$event, dataset$event)
print(evaluation$confMatrix)
## detection TRUE FALSE
## TRUE 1 0
## FALSE 0 100

har_plot(model, dataset$serie,
  detection, dataset$event)
har_plot(model,
  attr(detection, "res"),
  detection, dataset$event,
  yline = attr(detection,
    "threshold"))
```

Step by Step – Conv1D

- Convolutional filters capture local temporal patterns
- Translation invariance: detects patterns regardless of their position in the series
- Same `input_size` and `epochs` as LSTM
- Perfect result: 1 TP, 0 FP
- More efficient than LSTM for locally-structured anomalies

📌 **Key concept:** Conv1D applies sliding filters over the time window, capturing local patterns with translation invariance. It achieves a perfect result on this dataset — more efficient than LSTM when anomalies are defined by local shape rather than long-range dependencies.

Summary: Regression-Based Anomaly Detection

hanr_ml() – The Regression Wrapper

Accepts any DALToolbox regressor. Trains on normal behavior only — no labels required. Anomaly score = prediction residual. Residuals above a learned threshold are flagged as anomalies.

- **Packages required:** `harbinger`, `daltoolbox`, `tspredict` (+ `daltoolboxdp` for deep models)

Regressors Compared

- **ELM:** 1 FN, 1 FP — fast but less accurate
- **MLP:** 1 TP, 0 FP ✓ — best classical result
- **Random Forest:** 0 TP, 2 FP — sensitive to local variation
- **SVM (RBF):** 1 TP, 0 FP ✓ — robust non-linear mapping
- **LSTM:** 1 TP, 1 FP — captures long-range dependencies
- **Conv1D:** 1 TP, 0 FP ✓ — best for local shape anomalies

When to Use Each

- **No labels available** → use `hanr_ml()` with any regressor
- **Speed priority** → `ELM` (fixed random weights, $O(n)$ training)
- **Accuracy priority** → `SVM RBF` or `Conv1D` (1 TP, 0 FP)
- **Complex temporal patterns** → `LSTM` (long-range dependencies)
- **Local shape anomalies** → `Conv1D` (translation invariance)
- **Baseline comparison** → always start with `ELM` or `MLP`

Additional Sources

CRAN: harbinger

cran.r-project.org/web/packages/harbinger

Core package for anomaly detection in time series. Provides the `hanr_ml()` wrapper used throughout this deck.

CRAN: daltoolbox

cran.r-project.org/web/packages/daltoolbox

DAL Toolbox: base framework for ML models, normalization, and evaluation. Required by all regression detectors.

CRAN: tspredit

cran.r-project.org/web/packages/tspredit

Time series prediction models: `ts_elm()`, `ts_mlp()`, `ts_rf()`, `ts_svm()`.
Required for ELM, MLP, RF, and SVM slides.

CRAN: daltoolboxdp

cran.r-project.org/web/packages/daltoolboxdp

Deep learning extension: `ts_lstm()` and `ts_conv1d()`. Required for LSTM and Conv1D slides.

Event Detection Resources

eic.cefet-rj.br/~eogasawara/en/event-detection-in-time-series

Full tutorial and documentation by Eduardo Ogasawara. Covers regression-based and other event detection methods.

Issues & Discussions

github.com/cefet-rj-dal/harbinger/issues

Open questions, report bugs, or suggest improvements directly in the repository.